

# ME314 Homework 5

## Submission instructions

Deliverables that should be included with your submission are shown in **bold** at the end of each problem statement and the corresponding supplemental material. **Your homework will be graded IFF you submit a single PDF, .mp4 videos of animations when requested and a link to a Google colab file that meet all the requirements outlined below.**

- List the names of students you've collaborated with on this homework assignment.
- Include all of your code (and handwritten solutions when applicable) used to complete the problems.
- Highlight your answers (i.e. **bold** and outline the answers) for handwritten or markdown questions and include simplified code outputs (e.g. `.simplify()`) for python questions.
- Enable Google Colab permission for editing

- Click Share in the upper right corner
- Under "Get Link" click "Share with..." or "Change"
- Then make sure it says "Anyone with Link" and "Editor" under the dropdown menu

- Make sure all cells are run before submitting (i.e. check the permission by running your code in a private mode)

- Please don't make changes to your file after submitting, so we can grade it!

- Submit a link to your Google Colab file that has been run (before the submission deadline) and don't edit it afterwards!

**NOTE:** This Jupyter Notebook file serves as a template for you to start homework. Make sure you first copy this template to your own Google driver (click "File" -> "Save a copy in Drive"), and then start to edit it.

```
In [37]: #Import cell
import sympy as sym
import numpy as np
import matplotlib.pyplot as plt
```

```
In [38]: #####
# If you're using Google Colab, uncomment this section by selecting the whole section and press
# ctrl+'/' on your and keyboard. Run it before you start programming, this will enable the nice
# LaTeX "display()" function for you. If you're using the local Jupyter environment, leave it alone
#####
def custom_latex_printer(exp,**options):
    from google.colab.output._publish import javascript
    url = "https://cdnjs.cloudflare.com/ajax/libs/mathjax/3.1.1/latest.js?config=TeX-AMS_HTML"
    javascript(url=url)
    return sym.printing.latex(exp,**options)
sym.init_printing(use_latex="mathjax", latex_printer=custom_latex_printer)
```

Below are the help functions in previous homeworks, which you may need for this homework.

```
In [67]: def integrate(f, xt, dt):
        """
        This function takes in an initial condition x(t) and a timestep dt,
        as well as a dynamical system f(x) that outputs a vector of the
        same dimension as x(t). It outputs a vector x(t+dt) at the future
        time step.

        Parameters
        =====
        dyn: Python function
            derivate of the system at a given step x(t),
            it can considered as  $\dot{x}(t) = \text{func}(x(t))$ 
        xt: NumPy array
            current step x(t)
        dt:
            step size for integration

        Return
        =====
        new_xt:
            value of x(t+dt) integrated from x(t)
        """
        k1 = dt * f(xt)
        k2 = dt * f(xt+k1/2.)
        k3 = dt * f(xt+k2/2.)
        k4 = dt * f(xt+k3)
        new_xt = xt + (1/6.) * (k1+2.0*k2+2.0*k3+k4)
        return new_xt

def simulate(f, x0, tspan, dt, integrate):
    """
    This function takes in an initial condition x0, a timestep dt,
```

a time span `tspan` consisting of a list `[min_time, max_time]`, as well as a dynamical system `f(x)` that outputs a vector of the same dimension as `x0`. It outputs a full trajectory simulated over the time span of dimensions `(xvec_size, time_vec_size)`.

Parameters

=====

`f`: Python function

derivate of the system at a given step `x(t)`,  
it can considered as  $\dot{x}(t) = \text{func}(x(t))$

`x0`: NumPy array

initial conditions

`tspan`: Python list

`tspan = [min_time, max_time]`, it defines the start and end  
time of simulation

`dt`:

time step for numerical integration

`integrate`: Python function

numerical integration method used in this simulation

Return

=====

`x_traj`:

simulated trajectory of `x(t)` from `t=0` to `tf`

"""

`N = int((max(tspan)-min(tspan))/dt)`

`x = np.copy(x0)`

`tvec = np.linspace(min(tspan),max(tspan),N)`

`xtraj = np.zeros((len(x0),N))`

**for** `i` **in** `range(N)`:

`xtraj[:,i]=integrate(f,x,dt)`

`x = np.copy(xtraj[:,i])`

**return** `xtraj`

**def** `animate_double_pend(theta_array,L1=1,L2=1,T=10)`:

"""

Function to generate web-based animation of double-pendulum system

Parameters:

=====

`theta_array`:

trajectory of `theta1` and `theta2`, should be a NumPy array with  
shape of `(2,N)`

`L1`:

length of the first pendulum

`L2`:

length of the second pendulum

`T`:

length/seconds of animation duration

Returns: None

"""

#####

*# Imports required for animation.*

**from** `plotly.offline` **import** `init_notebook_mode`, `ipplot`

**from** `IPython.display` **import** `display`, `HTML`

**import** `plotly.graph_objects` **as** `go`

#####

*# Browser configuration.*

**def** `configure_plotly_browser_state()`:

**import** `IPython`

`display(IPython.core.display.HTML(''`

`<script src="/static/components/requirejs/require.js"></script>`

`<script>`

`requirejs.config({`

`paths: {`

`base: '/static/base',`

`plotly: 'https://cdn.plot.ly/plotly-latest.min.js?noext',`

`},`

`});`

`</script>`

`''))`

`configure_plotly_browser_state()`

`init_notebook_mode(connected=False)`

#####

*# Getting data from pendulum angle trajectories.*

`xx1=L1*np.sin(theta_array[0])`

`yy1=-L1*np.cos(theta_array[0])`

`xx2=xx1+L2*np.sin(theta_array[0]+theta_array[1])`

`yy2=yy1-L2*np.cos(theta_array[0]+theta_array[1])`

`N = len(theta_array[0])` *# Need this for specifying length of simulation*

#####

*# Using these to specify axis limits.*

```

xm=np.min(xx1)-0.5
xM=np.max(xx1)+0.5
ym=np.min(yy1)-2.5
yM=np.max(yy1)+1.5

#####
# Defining data dictionary.
# Trajectories are here.
data=[dict(x=xx1, y=yy1,
           mode='lines', name='Arm',
           line=dict(width=2, color='blue')
          ),
      dict(x=xx1, y=yy1,
           mode='lines', name='Mass 1',
           line=dict(width=2, color='purple')
          ),
      dict(x=xx2, y=yy2,
           mode='lines', name='Mass 2',
           line=dict(width=2, color='green')
          ),
      dict(x=xx1, y=yy1,
           mode='markers', name='Pendulum 1 Traj',
           marker=dict(color="purple", size=2)
          ),
      dict(x=xx2, y=yy2,
           mode='markers', name='Pendulum 2 Traj',
           marker=dict(color="green", size=2)
          ),
      ]

#####
# Preparing simulation layout.
# Title and axis ranges are here.
layout=dict(xaxis=dict(range=[xm, xM], autorange=False, zeroline=False,dtick=1),
            yaxis=dict(range=[ym, yM], autorange=False, zeroline=False,scaleanchor = "x",dtick=1),
            title='Triple Pendulum Simulation',
            hovermode='closest',
            updatemenus= [{ 'type': 'buttons',
                           'buttons': [{ 'label': 'Play', 'method': 'animate',
                                           'args': [None, {'frame': {'duration': T, 'redraw': False}}]},
                                         { 'args': [[None], {'frame': {'duration': T, 'redraw': False}, 'mode': 'i',
                                           'transition': {'duration': 0}}], 'label': 'Pause', 'method': 'animate'}
                                       ]
                         }
            ])

#####
# Defining the frames of the simulation.
# This is what draws the lines from
# joint to joint of the pendulum.
frames=[dict(data=[dict(x=[0,xx1[k],xx2[k]],
                        y=[0,yy1[k],yy2[k]],
                        mode='lines',
                        line=dict(color='red', width=3)
                        ),
                go.Scatter(
                    x=[xx1[k]],
                    y=[yy1[k]],
                    mode="markers",
                    marker=dict(color="blue", size=12)),
                go.Scatter(
                    x=[xx2[k]],
                    y=[yy2[k]],
                    mode="markers",
                    marker=dict(color="blue", size=12)),
            ]) for k in range(N)]

#####
# Putting it all together and plotting.
figure1=dict(data=data, layout=layout, frames=frames)
iplot(figure1)

def animate_triple_pend(theta_array, L1=1, L2=1, L3=1, T=10):
    """
    Function to generate web-based animation of triple-pendulum system

    Parameters:
    =====
    theta_array:
        trajectory of theta1 and theta2, should be a NumPy array with
        shape of (3,N)
    L1:
        length of the first pendulum
    L2:
        length of the second pendulum
    L3:
        length of the third pendulum
    """

```

```

T:
    length/seconds of animation duration

Returns: None
"""

#####
# Imports required for animation.
from plotly.offline import init_notebook_mode, iplot
from IPython.display import display, HTML
import plotly.graph_objects as go

#####
# Browser configuration.
def configure_plotly_browser_state():
    import IPython
    display(IPython.core.display.HTML('''
        <script src="/static/components/requirejs/require.js"></script>
        <script>
            requirejs.config({
                paths: {
                    base: '/static/base',
                    plotly: 'https://cdn.plot.ly/plotly-1.5.1.min.js?noext',
                },
            });
        </script>
        '''))
configure_plotly_browser_state()
init_notebook_mode(connected=False)

#####
# Getting data from pendulum angle trajectories.
xx1=L1*np.sin(theta_array[0])
yy1=-L1*np.cos(theta_array[0])
xx2=xx1+L2*np.sin(theta_array[0]+theta_array[1])
yy2=yy1-L2*np.cos(theta_array[0]+theta_array[1])
xx3=xx2+L3*np.sin(theta_array[0]+theta_array[1]+theta_array[2])
yy3=yy2-L3*np.cos(theta_array[0]+theta_array[1]+theta_array[2])
N = len(theta_array[0]) # Need this for specifying length of simulation

#####
# Using these to specify axis limits.
xm=np.min(xx1)-0.5
xM=np.max(xx1)+0.5
ym=np.min(yy1)-2.5
yM=np.max(yy1)+1.5

#####
# Defining data dictionary.
# Trajectories are here.
data=[dict(x=xx1, y=yy1,
            mode='lines', name='Arm',
            line=dict(width=2, color='blue')
            ),
      dict(x=xx1, y=yy1,
            mode='lines', name='Mass 1',
            line=dict(width=2, color='purple')
            ),
      dict(x=xx2, y=yy2,
            mode='lines', name='Mass 2',
            line=dict(width=2, color='green')
            ),
      dict(x=xx3, y=yy3,
            mode='lines', name='Mass 3',
            line=dict(width=2, color='yellow')
            ),
      dict(x=xx1, y=yy1,
            mode='markers', name='Pendulum 1 Traj',
            marker=dict(color="purple", size=2)
            ),
      dict(x=xx2, y=yy2,
            mode='markers', name='Pendulum 2 Traj',
            marker=dict(color="green", size=2)
            ),
      dict(x=xx3, y=yy3,
            mode='markers', name='Pendulum 3 Traj',
            marker=dict(color="yellow", size=2)
            ),
      ]

#####
# Preparing simulation layout.
# Title and axis ranges are here.
layout=dict(xaxis=dict(range=[xm, xM], autorange=False, zeroline=False,dtick=1),
            yaxis=dict(range=[ym, yM], autorange=False, zeroline=False,scaleanchor = "x",dtick=1),
            title='Triple Pendulum Simulation',
            hovermode='closest',

```

```

        updatemenus= [{ 'type': 'buttons',
                        'buttons': [{ 'label': 'Play', 'method': 'animate',
                                      'args': [None, {'frame': {'duration': T, 'redraw': False}}]},
                                   {'args': [[None], {'frame': {'duration': T, 'redraw': False}, 'mode': 'i',
                                                              'transition': {'duration': 0}}]}, 'label': 'Pause', 'method': 'animate'}
                        ]
        })

#####
# Defining the frames of the simulation.
# This is what draws the lines from
# joint to joint of the pendulum.
frames=[dict(data=[dict(x=[0,xx1[k],xx2[k],xx3[k]],
                        y=[0,yy1[k],yy2[k],yy3[k]],
                        mode='lines',
                        line=dict(color='red', width=3)
                        ),
                go.Scatter(
                    x=[xx1[k]],
                    y=[yy1[k]],
                    mode="markers",
                    marker=dict(color="blue", size=12)),
                go.Scatter(
                    x=[xx2[k]],
                    y=[yy2[k]],
                    mode="markers",
                    marker=dict(color="blue", size=12)),
                go.Scatter(
                    x=[xx3[k]],
                    y=[yy3[k]],
                    mode="markers",
                    marker=dict(color="blue", size=12)),
                ]) for k in range(N)]

#####
# Putting it all together and plotting.
figure1=dict(data=data, layout=layout, frames=frames)
iplot(figure1)

```

```

In [40]: def animate_single_pend(theta_array, L=1, T=10):
        """
        Function to generate web-based animation of a single-pendulum system

        Parameters:
        =====
        theta_array:
            trajectory of theta, should be a NumPy array with shape (N,)
        L:
            length of the pendulum
        T:
            length/seconds of animation duration

        Returns: None
        """

        #####
        # Imports required for animation.
        from plotly.offline import init_notebook_mode, iplot
        from IPython.display import display, HTML
        import plotly.graph_objects as go

        #####
        # Browser configuration.
        def configure_plotly_browser_state():
            import IPython
            display(IPython.core.display.HTML('''
                <script src="/static/components/requirejs/require.js"></script>
                <script>
                    requirejs.config({
                        paths: {
                            base: '/static/base',
                            plotly: 'https://cdn.plot.ly/plotly-latest.min.js?noext',
                        },
                    });
                </script>
            '''))
        configure_plotly_browser_state()
        init_notebook_mode(connected=False)

        #####
        # Getting data from pendulum angle trajectory.
        xx = L * np.sin(theta_array)
        yy = -L * np.cos(theta_array)
        N = len(theta_array) # Length of simulation

        #####

```

```

# Using these to specify axis limits.
xm = np.min(xx) - 0.5
xM = np.max(xx) + 0.5
ym = np.min(yy) - 1.5
yM = np.max(yy) + 0.5

#####
# Defining data dictionary.
data = [
    dict(x=xx, y=yy, mode='lines', name='Pendulum Arm', line=dict(width=2, color='blue')),
    dict(x=xx, y=yy, mode='markers', name='Pendulum Mass Traj', marker=dict(color="blue", size=2)),
]

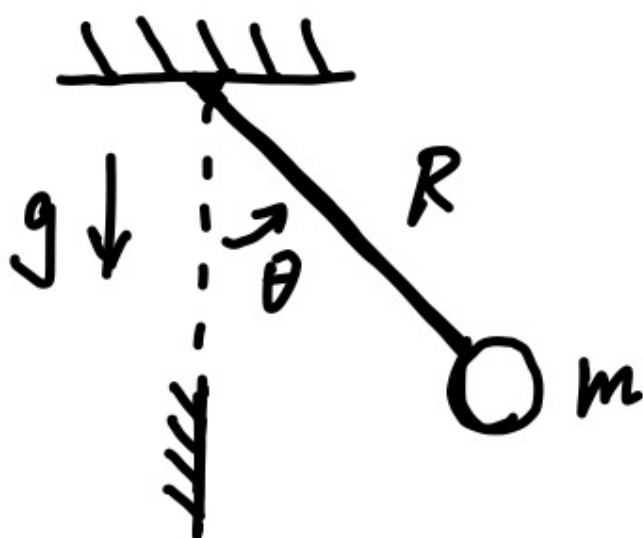
#####
# Preparing simulation layout.
layout = dict(
    xaxis=dict(range=[xm, xM], autorange=False, zeroline=False, dtick=1),
    yaxis=dict(range=[ym, yM], autorange=False, zeroline=False, scaleanchor="x", dtick=1),
    title='Single Pendulum Simulation',
    hovermode='closest',
    updatemenus=[{'type': 'buttons',
                  'buttons': [{'label': 'Play', 'method': 'animate',
                              'args': [None, {'frame': {'duration': T, 'redraw': False}}]},
                              {'args': [[None], {'frame': {'duration': T, 'redraw': False},
                                                    'mode': 'immediate', 'transition': {'duration': 0}}]},
                              {'label': 'Pause', 'method': 'animate'}]}]
)

#####
# Defining the frames of the simulation.
frames = [dict(data=[dict(x=[0, xx[k]], y=[0, yy[k]], mode='lines', line=dict(color='red', width=3)),
                    go.Scatter(x=[xx[k]], y=[yy[k]], mode="markers", marker=dict(color="blue", size=12)),
                    ]) for k in range(N)]

#####
# Putting it all together and plotting.
figure1 = dict(data=data, layout=layout, frames=frames)
iplot(figure1)

```

In [41]: `from IPython.core.display import HTML`  
`display(HTML("<table><tr><td><img src='https://github.com/MuchenSun/ME314pngs/raw/master/singlepend.JPG' width=350"`



## Problem 1 (5pts)

Consider the single pendulum showed above. Solve the Euler-Lagrange equations and simulate the system for  $t \in [0, 5]$  with  $dt = 0.01$ ,  $R = 1$ ,  $m = 1$ ,  $g = 9.8$  given initial condition as  $\theta = \frac{\pi}{2}$ ,  $\dot{\theta} = 0$ . Plot your simulation of the system (i.e.  $\theta$  versus time). Note that in this problem there is no impact involved (ignore the wall at the bottom).

**Turn in:** A copy of the code used to solve the EL-equations and numerically simulate the system. Also include code output, which should be the plot of the trajectory versus time.

In [42]: `# Define the variables and parameters`  
`t, g, m, R = sym.symbols('t, g, m, R')`  
`th = sym.Function('theta')(t)`  
`th_dot = th.diff(t)`  
`x = R*sym.sin(th)`  
`y = -R*sym.cos(th)`



```
x_dot = x.diff(t)
y_dot = y.diff(t)

KE = (m*(x_dot**2 + y_dot**2))/2
PE = m*g*y

lagrangian = KE - PE
lagrangian = sym.simplify(lagrangian)

# Display Lagrangian
print('\033[1mLagrangian of the system: ')
display(lagrangian)
print("\n")

# Wrap the lagrangian into a SymPy Matrix
L = sym.Matrix([lagrangian])
# display(L)

q = sym.Matrix([th])
qdot = q.diff(t)
qddot = qdot.diff(t)

# Compute Jacobian
dLdq = L.jacobian(q)
dLdqdot = L.jacobian(qdot)

# print('\033[1mEuler Lagrangian Equations')
EL = dLdqdot.diff(t) - dLdq

# for exp, n in zip(EL, range(1,3)):
#     print("\033[1mEL Equation {}".format(n))
#     display(sym.Eq(exp, 0))
#     print("\n")

# LHS of the EL equations
lhs = sym.Matrix([exp for exp in EL])

# RHS of the EL equations
rhs = sym.Matrix([0 for exp in EL])

# Define the equations
eqns = sym.Eq(lhs, rhs)
print('\033[1mEuler Lagrangian Equations')
display(eqns)

solns = sym.solve(eqns, qddot, dict=True)

for sol in solns:
    print('\n\033[1mSolution: ')
    for q_ in qddot:
        display(sym.simplify(sym.Eq(q_, sol[q_])))
        print("\n")

# Substitute constants into symbolic solutions
subs_dict = {m:1, g:9.8, R:1}

c_sub_solns = [{q_: soln[q_].subs(subs_dict) for soln in solns for q_ in qddot}]

for c_sub_soln in c_sub_solns:
    print('\n\033[1mSolution: ')
    for q_ in qddot:
        display(sym.simplify(sym.Eq(q_, c_sub_soln[q_])))
        print("\n")

# Lamdifying the constant substituted solutions for theta1_ddot and theta2_ddot
state_vec = [th]
lambdified_solns = [sym.lambdify(state_vec, c_sub_soln[q_]) for c_sub_soln in c_sub_solns for q_ in qddot]
thddot_func, = lambdified_solns
```

**Lagrangian of the system:**

$$\frac{Rm \left( R \left( \frac{d}{dt} \theta(t) \right)^2 + 2g \cos(\theta(t)) \right)}{2}$$

**Euler Lagrangian Equations**

$$\left[ R^2 m \frac{d^2}{dt^2} \theta(t) + R g m \sin(\theta(t)) \right] = [0]$$

**Solution:**

$$\frac{d^2}{dt^2} \theta(t) = - \frac{g \sin(\theta(t))}{R}$$

**Solution:**



$$\frac{d^2}{dt^2}\theta(t) = -9.8 \sin(\theta(t))$$

```
In [43]: # define extended dynamics for the pendulum
def pendulum_dyn(x):
    """
    System dynamics function (extended)

    Parameters
    =====
    s: NumPy array
    s = [th, thdot] is the extended system
    state vector, including the position and
    the velocity of the particle

    Return
    =====
    sdot: NumPy array
    time derivative of input state vector,
    sdot = [thdot, thddot]

    """
    theta_k = x[0]
    thetadot_k = x[1]

    thetaddot_k = thddot_func(theta_k)

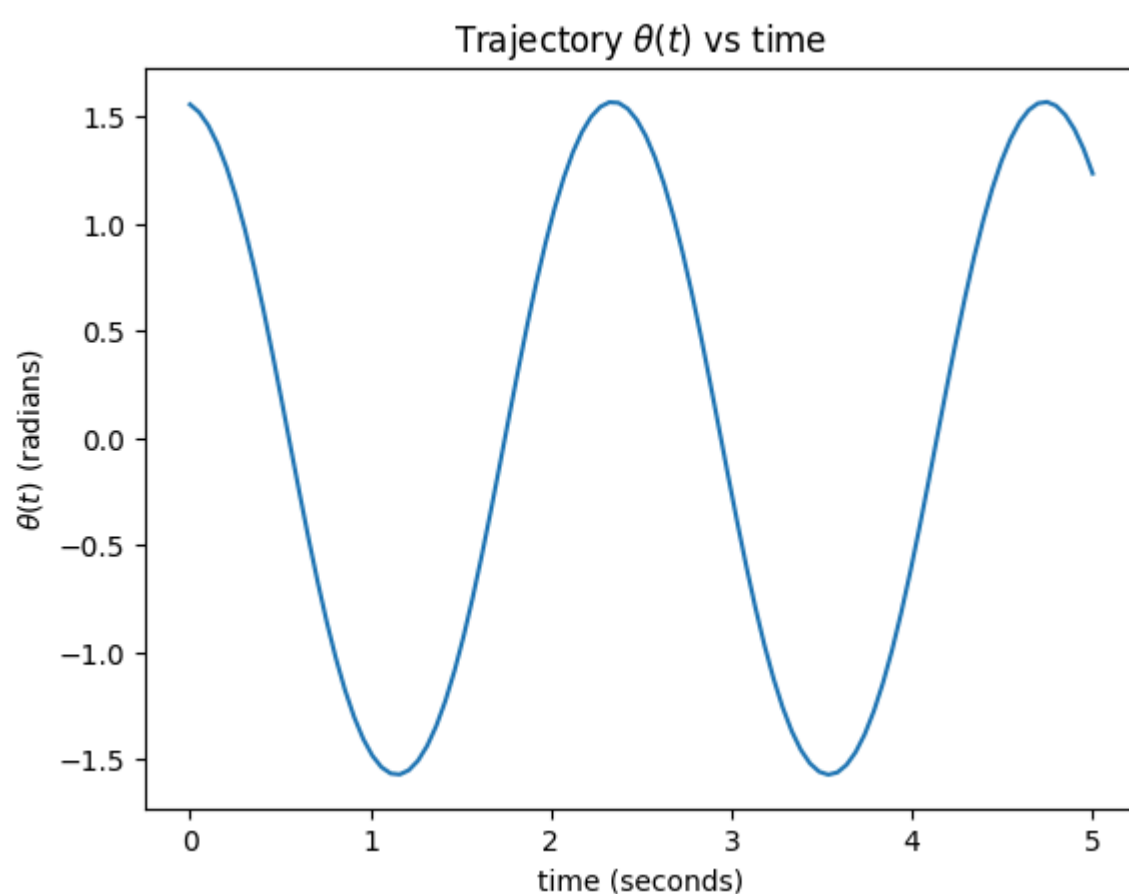
    return np.array([thetadot_k, thetaddot_k])

# Simulation parameters
t0 = 0
tf = 5
dt = 0.05
N = int((tf - t0)/dt)
tvec = np.linspace(t0, tf, N)

# initial conditions
x0 = np.array([np.pi/2, 0])

# simulate the trajectory
x_traj = simulate(pendulum_dyn, x0, [t0, tf], dt, integrate)

# plot th(t)
plt.plot(tvec, x_traj[0,:])
plt.title(r'Trajectory $\theta(t)$ vs time')
plt.xlabel("time (seconds)")
plt.ylabel(r"$\theta(t)$ (radians)")
plt.show()
```



## Problem 2 (10pts)

Now, time for impact (i.e. don't ignore the vertical wall) ! As shown in the figure above, there is a wall such that the pendulum will hit it when  $\theta = 0$ . Recall that in the course notes, to solve the impact update rule, we have two set of equations:



$$\left. \frac{\partial L}{\partial \dot{q}} \right|_{\tau^-}^{\tau^+} = \lambda \frac{\partial \phi}{\partial q}$$

$$\left[ \frac{\partial L}{\partial \dot{q}} \cdot \dot{q} - L(q, \dot{q}) \right] \bigg|_{\tau^-}^{\tau^+} = 0$$

In this problem, you will need to symbolically compute the following three expressions contained the equations above:

$$\frac{\partial L}{\partial \dot{q}}, \quad \frac{\partial \phi}{\partial q}, \quad \frac{\partial L}{\partial \dot{q}} \cdot \dot{q} - L(q, \dot{q})$$

Hint 1: The third expression is the Hamiltonian of the system.

Hint 2: All three expressions can be considered as functions of  $q$  and  $\dot{q}$ . If you have previously defined  $q$  and  $\dot{q}$  as SymPy's function objects, now you will need to substitute them with dummy symbols (using SymPy's substitute method)

Hint 3:  $q$  and  $\dot{q}$  should be two sets of separate symbols.

**Turn in: A copy of code used to symbolically compute the three expressions, also include the outputs of your code, which should be the three expressions (make sure there is no SymPy Function(t) left in your solution output).**

```
In [44]: # Define the phi(q)
phi = q[0] - 0
print("Phi")
display(phi)
print("\n")

gradphi = sym.simplify(sym.Matrix([phi]).jacobian(q).T)
print("Gradient of the surface along q:")
display(gradphi)
print("\n")

# Compute dL/dqdot
print("dL/dqdot")
display(dLdqqdot)
print("\n")

# Define the hamiltonian
hamiltonian = sym.simplify(dLdqqdot.T.dot(qdot)-lagrangian)
print("Hamiltonian:")
display(hamiltonian)
```

Phi

$\theta(t)$

Gradient of the surface along q:

[1]

dL/dqdot

$$\left[ R^2 m \frac{d}{dt} \theta(t) \right]$$

Hamiltonian:

$$\frac{Rm \left( R \left( \frac{d}{dt} \theta(t) \right)^2 - 2g \cos(\theta(t)) \right)}{2}$$

```
In [45]: tau, tau_minus, tau_plus, lambda_ = sym.symbols(r'\tau, \tau^-, \tau^+, \lambda')
```

```
# Configuration variables before and after impact
q_tau = q.subs(t,tau)
q_minus = q.subs(t,tau_minus)
q_plus = q.subs(t,tau_plus)
qdot_minus = qdot.subs(q[0], q_minus[0])
qdot_plus = qdot.subs(q[0], q_plus[0])

# display(q)
# display(qdot)
# display(sym.simplify(q_minus - q_plus))
# display(qdot_minus)
# display(qdot_plus)

# display((qdot_minus - qdot_plus))
# display(sym.simplify(qdot_minus - qdot_plus))
```

```
In [46]: # Momentum before and after impact
p_minus = dLdqqdot.subs({q[0]:q_minus[0],qdot[0]:qdot_minus[0]}).T
p_plus = dLdqqdot.subs({q[0]:q_plus[0],qdot[0]:qdot_plus[0]}).T
p_eq = sym.Eq((p_plus-p_minus),lambda_*gradphi)

print("Momentum evaluated before and after impact")
display(p_minus)
```

```

display(p_plus)
print("\n")

print("Impact equations due to momentum terms:")
display(p_eq)
print("\n")

hamiltonian_minus = hamiltonian.subs({q[0]:q_minus[0],qdot[0]:qdot_minus[0]})
hamiltonian_plus = hamiltonian.subs({q[0]:q_plus[0],qdot[0]:qdot_plus[0]})

print("Hamiltonian evaluated before and after impact:")
display(hamiltonian_minus)
display(hamiltonian_plus)
print("\n")
# display(sym.simplify(hamiltonian_plus - hamiltonian_minus))

print("Impact equations due to hamiltonian terms:")
display(sym.Eq((hamiltonian_plus - hamiltonian_minus), 0))

```

Momentum evaluated before and after impact

$$\left[ R^2 m \frac{d}{dt} \theta(\tau^-) \right]$$

$$\left[ R^2 m \frac{d}{dt} \theta(\tau^+) \right]$$

Impact equations due to momentum terms:

$$\left[ R^2 m \frac{d}{dt} \theta(\tau^+) - R^2 m \frac{d}{dt} \theta(\tau^-) \right] = [\lambda]$$

Hamiltonian evaluated before and after impact:

$$\frac{Rm \left( R \left( \frac{d}{dt} \theta(\tau^-) \right)^2 - 2g \cos(\theta(\tau^-)) \right)}{2}$$

$$\frac{Rm \left( R \left( \frac{d}{dt} \theta(\tau^+) \right)^2 - 2g \cos(\theta(\tau^+)) \right)}{2}$$

Impact equations due to hamiltonian terms:

$$\frac{Rm \left( R \left( \frac{d}{dt} \theta(\tau^+) \right)^2 - 2g \cos(\theta(\tau^+)) \right)}{2} - \frac{Rm \left( R \left( \frac{d}{dt} \theta(\tau^-) \right)^2 - 2g \cos(\theta(\tau^-)) \right)}{2} = 0$$

In [47]:

```

ham_minus_actual = hamiltonian.subs({qdot[0]:qdot_minus[0]})
ham_plus_actual = hamiltonian.subs({qdot[0]:qdot_plus[0]})
ham_eq = sym.Eq((ham_minus_actual-ham_plus_actual),0)
display(ham_eq)

```

$$-\frac{Rm \left( R \left( \frac{d}{dt} \theta(\tau^+) \right)^2 - 2g \cos(\theta(t)) \right)}{2} + \frac{Rm \left( R \left( \frac{d}{dt} \theta(\tau^-) \right)^2 - 2g \cos(\theta(t)) \right)}{2} = 0$$

## Problem 3 (10pts)

Now everything is ready for you to solve the impact update rules! To solve those equations, you will need to evaluate them right before and after the impact time at  $\tau^-$  and  $\tau^+$ .

Hint 1: Here  $\dot{q}(\tau^-)$  is actually same as the dummy symbol you defined in Problem 2 (why?), but you will need to define new dummy symbol for  $\dot{q}(\tau^+)$ . That is to say,  $\frac{\partial L}{\partial \dot{q}}$  and  $\frac{\partial L}{\partial \dot{q}} \cdot \dot{q} - L(q, \dot{q})$  evaluated at  $\tau^-$  are those you already had in Problem 2, but you will need to substitute the dummy symbols of  $\dot{q}(\tau^+)$  to evaluate them at  $\tau^+$ .

Based on the information above, define the equations for impact update and solve them for impact update rules. After solving the impact update solution, numerically evaluate it as a function using SymPy's lambdify method and test it with  $\theta(\tau^-) = 0.01, \dot{\theta}(\tau^-) = 2$ .

Hint 2: In your equations and impact update solutions, there should be NO SymPy Function left (except for internal functions like sin or cos).

Hint 3: You may wonder where are  $q(\tau^-)$  and  $q(\tau^+)$ ? The real question at hand is do we really need new dummy variables for them?

Hint 4: The solution of the impact update rules, which is obtained by solving the equations for the dummy variables corresponds to  $\dot{q}(\tau^+)$  and  $\lambda$ , can be a function of  $q(\tau^-)$  or a function of  $q(\tau^-)$  and  $\dot{q}(\tau^-)$ . While  $q$  will not be updated during impact, including it now (as an argument in your lambdify function) may help you to combine the function into simulation later.

**Turn in: A copy of code used to symbolically solve for the impact update rules and evaluate them numerically. Also, include the outputs of your code, which should be the test output of your numerically evaluated impact update function.**

```
In [48]: ImpactEqs = sym.Eq(sym.Matrix([ham_eq.lhs,p_eq.lhs[0]]),sym.Matrix([ham_eq.rhs,p_eq.rhs[0]]))
ImpactSols = sym.solve(ImpactEqs,[qdot_plus[0],lambda_])

# display(ImpactSols)
print("Solution 1:")
sol1 = sym.Matrix([ImpactSols[0][0],ImpactSols[0][1]])
display(sym.Eq(sym.Matrix([qdot_plus[0],lambda_]),sol1))

print("Solution 2:")
sol2 = sym.Matrix([ImpactSols[1][0],ImpactSols[1][1]])
display(sym.Eq(sym.Matrix([qdot_plus[0],lambda_]),sol2))
print("\n")

# We will take Solution 2 (Solution where velocities change before and after impact)

# Substitute constants into symbolic solutions
subs_dict = {m:1, g:9.8, R:1}
sol2 = sym.Matrix([sol_.subs(subs_dict) for sol_ in sol2])
print("Substituting the constant values in solution")
display(sym.Eq(sym.Matrix([qdot_plus[0],lambda_]),sol2))

state_vect = [q_minus[0], qdot_minus[0]]
# Lamdifying the constant substituted solutions for theta_ddot
lambdified_solns = [sym.lambdify(state_vect, sol_) for sol_ in sol2]
qdot_plus_func, lambda_func = lambdified_solns
```

Solution 1:

$$\begin{bmatrix} \frac{d}{dt}\theta(\tau^+) \\ \lambda \end{bmatrix} = \begin{bmatrix} \frac{d}{dt}\theta(\tau^-) \\ 0 \end{bmatrix}$$

Solution 2:

$$\begin{bmatrix} \frac{d}{dt}\theta(\tau^+) \\ \lambda \end{bmatrix} = \begin{bmatrix} -\frac{d}{dt}\theta(\tau^-) \\ -2R^2m\frac{d}{dt}\theta(\tau^-) \end{bmatrix}$$

Substituting the constant values in solution

$$\begin{bmatrix} \frac{d}{dt}\theta(\tau^+) \\ \lambda \end{bmatrix} = \begin{bmatrix} -\frac{d}{dt}\theta(\tau^-) \\ -2\frac{d}{dt}\theta(\tau^-) \end{bmatrix}$$

```
In [49]: theta_minus = 0.1
theta_dot_minus = 2

print(f"Evaluating qdot at tau plus : {qdot_plus_func(theta_minus, theta_dot_minus)}")
print(f"Evaluating lambda : {lambda_func(theta_minus, theta_dot_minus)}")
```

Evaluating qdot at tau plus : -2

Evaluating lambda : -4

## Problem 4 (20pts)

Finally, it's time to simulate the impact! To use impact update rules with our previous simulate function, there two more steps:

1. Write a function called "impact\_condition", which takes in  $s = [q, \dot{q}]$  and returns **True** if  $s$  will cause an impact, otherwise the function will return **False**.

Hint 1 : you need to use the constraint  $\phi$  in this problem, and note that, since we are doing numerical evaluation, the impact condition will not be perfect, you will need to catch the change of sign at  $\phi(s)$  or setup a threshold to decide the condition.

2. Now, with the "impact\_condition" function and the numerically evaluated impact update rule for  $\dot{q}(\tau^+)$  solved in last problem, find a way to combine them into the previous simulation function, thus it can simulate the impact. Pseudo-code for the simulate function can be found in lecture note 13.

Simulate the system with same parameters and initial condition in Problem 1 for the single pendulum hitting the wall for five times. Plot the trajectory and animate the simulation (you need to modify the animation function by yourself).

**Turn in: A copy of the code used to simulate the system. You don't need to include the animation function, but please include other code (impact\_condition, simulate, etc.) used for simulating impact. Also, include the plot and a video for animation. The video can be uploaded separately through Canvas, and it should be in ".mp4" format. You can use screen capture or record the screen directly with your phone.**

```
In [50]: phi_func = sym.lambdify([q[0], qdot[0]], phi)

def impact_condition(x, threshold=1e-1):
    q = x[0]
    qdot = x[1]
    phi_eval = phi_func(q, qdot)

    if(abs(phi_eval) < threshold):
        return True
```

```

    return False

def impact_update(x):
    q = x[0]
    qdot = x[1]

    return np.array([q, qdot_plus_func(q, qdot)])

# define simulate function
def simulate_with_impact(f, x0, tspan, dt, integrate):

    N = int((max(tspan) - min(tspan)) / dt)
    x = np.copy(x0)
    tvec = np.linspace(min(tspan), max(tspan), N)
    xtraj = np.zeros((len(x0), N))
    impact_eval = False
    for i in range(N):
        if impact_condition(x) is True and not impact_eval:
            x = impact_update(x)
            xtraj[:, i] = integrate(f, x, dt)
            impact_eval = True
        else:
            xtraj[:, i] = integrate(f, x, dt)
            impact_eval = False
        x = np.copy(xtraj[:, i])

    return xtraj

# Simulation parameters
t0 = 0
tf = 6
dt = 0.01
N = int((tf - t0)/dt)
tvec = np.linspace(t0, tf, N)

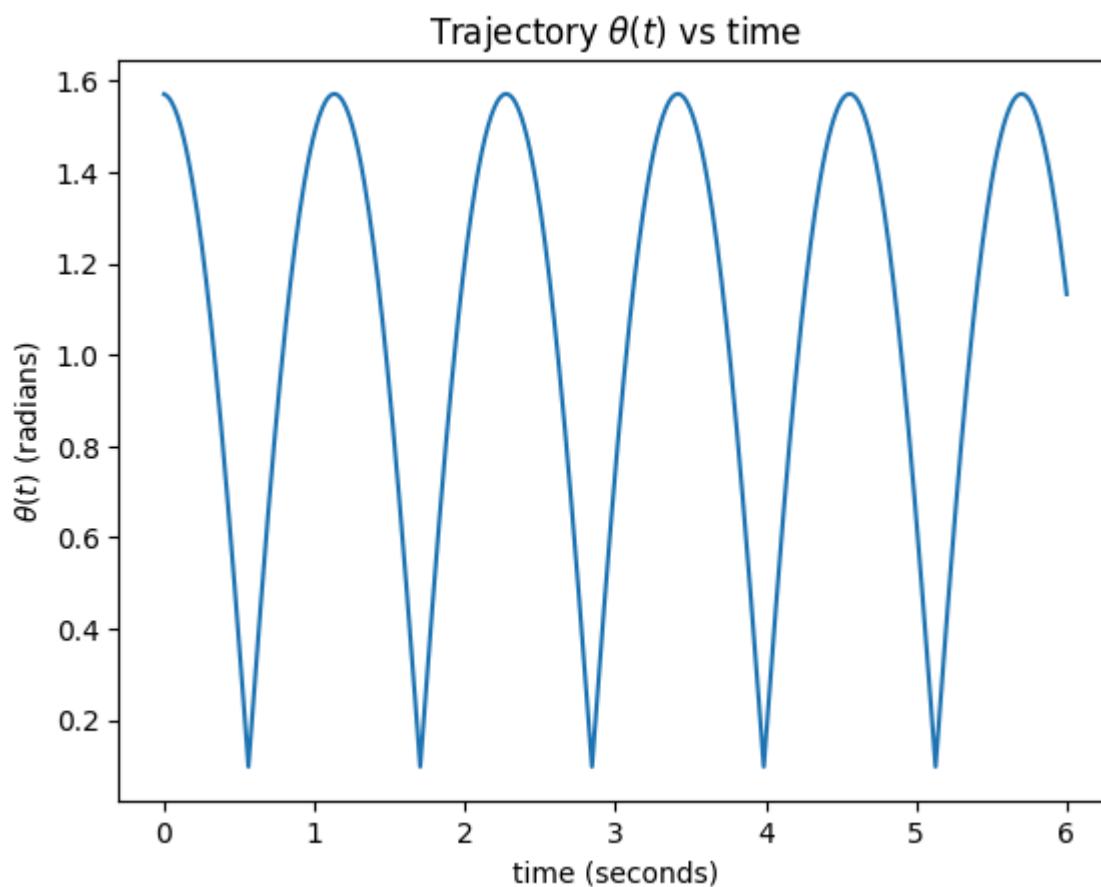
# Initial condition
x0 = [np.pi/2, 0]
x_traj = simulate_with_impact(pendulum_dyn, x0, tspan=[t0, tf], dt=dt, integrate=integrate)

```

```

In [51]: # plot  $\theta(t)$ 
plt.plot(tvec, x_traj[0,:])
plt.title(r'Trajectory  $\theta(t)$  vs time')
plt.xlabel("time (seconds)")
plt.ylabel(r" $\theta(t)$  (radians)")
plt.show()

```

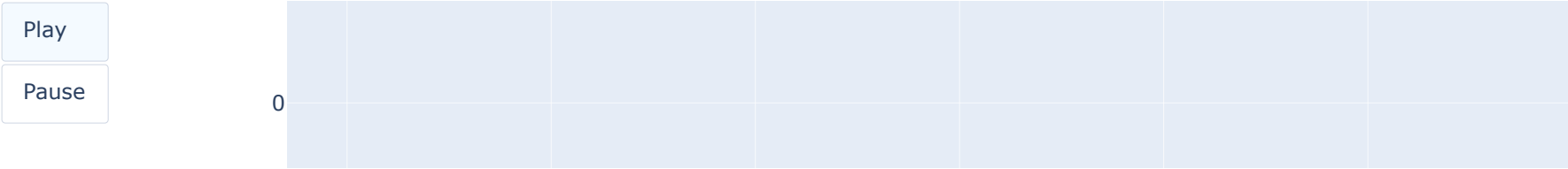


```

In [52]: animate_single_pend(x_traj[0,:], L=1, T=tf-t0)

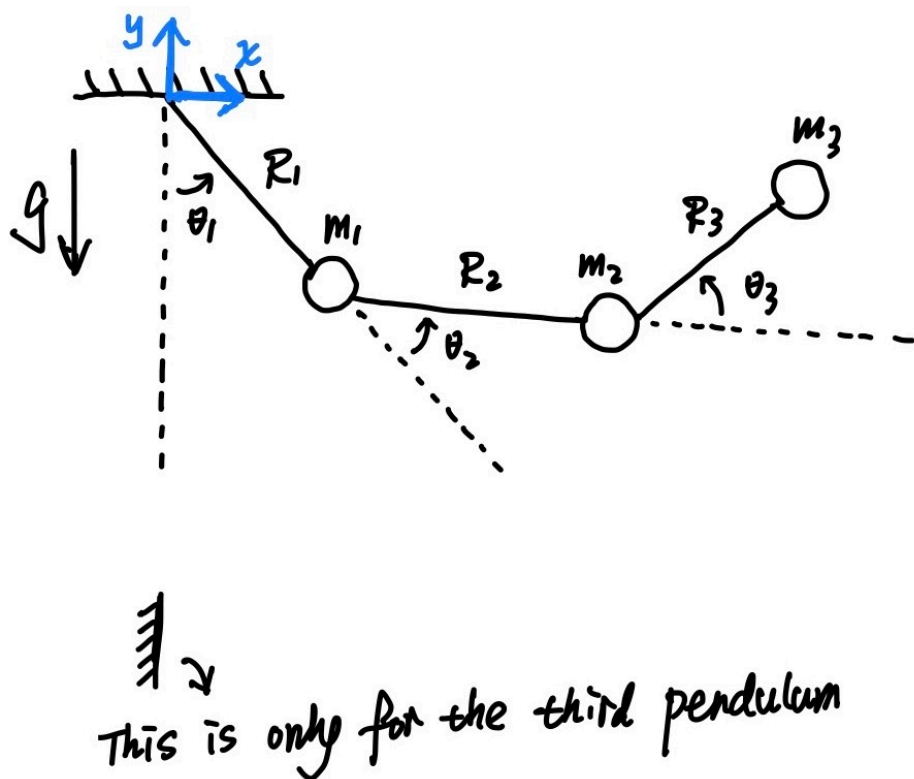
```

Single Pendulum Simulation



Problem 5 (10pts)

```
In [53]: from IPython.core.display import HTML
display(HTML("<table><tr><td><img src='https://github.com/MuchenSun/ME314pngs/raw/master/tripend_constrained.JPG' w
```



We will now consider a constrained triple-pendulum system with the system configuration  $q = [\theta_1, \theta_2, \theta_3]$ . A constraint is such that  $x$  coordinate of the third pendulum (i.e.  $m_3$ ) ONLY can not be smaller than 0 -- there exist a vertical wall high enough for third pendulum impact. Note that there is no constraint on  $y$  coordinate -- the top ceiling is infinitely high!

Similar to Problem 2, symbolically compute the following three expressions contained the equations above:

$$\frac{\partial L}{\partial \dot{q}}, \quad \frac{\partial \phi}{\partial q}, \quad \frac{\partial L}{\partial \dot{q}} \cdot \dot{q} - L(q, \dot{q})$$

Use  $m_1 = m_2 = m_3 = 1$  and  $R_1 = R_2 = R_3 = 1$  as numerical values in the equations (i.e. **do not** define  $m_1, m_2, m_3, R_1, R_2, R_3$  as symbols).

Hint 1: As before, you will need to substitute  $q$  and  $\dot{q}$  with dummy symbols.

**Turn in:** Include the code used to symbolically compute the three expressions, as well as code outputs - the resulting three expressions. Make sure there is no SymPy Function(t) left!

```
In [54]: # Define the variables and parameters
t = sym.symbols('t')
```

```

th1 = sym.Function(r'\theta1')(t)
th2 = sym.Function(r'\theta2')(t)
th3 = sym.Function(r'\theta3')(t)

th1_dot = th1.diff(t)
th2_dot = th2.diff(t)
th3_dot = th2.diff(t)

x1 = sym.sin(th1)
y1 = -sym.cos(th1)

x2 = x1 + sym.sin(th1 + th2)
y2 = y1 - sym.cos(th1 + th2)

x3 = x2 + sym.sin(th1 + th2 + th3)
y3 = y2 - sym.cos(th1 + th2 + th3)

x1_dot = x1.diff(t)
y1_dot = y1.diff(t)

x2_dot = x2.diff(t)
y2_dot = y2.diff(t)

x3_dot = x3.diff(t)
y3_dot = y3.diff(t)

KE1 = (x1_dot**2 + y1_dot**2)/2
PE1 = 9.8*y1

KE2 = (x2_dot**2 + y2_dot**2)/2
PE2 = 9.8*y2

KE3 = (x3_dot**2 + y3_dot**2)/2
PE3 = 9.8*y3

KE = KE1 + KE2 + KE3
PE = PE1 + PE2 + PE3

lagrangian = KE - PE
lagrangian = sym.simplify(lagrangian)

# Display Lagrangian
print('\033[1mLagrangian of the system: ')
display(lagrangian)
print("\n")

# Wrap the lagrangian into a SymPy Matrix
L = sym.Matrix([lagrangian])
# display(L)

q = sym.Matrix([th1, th2, th3])
qdot = q.diff(t)
qddot = qdot.diff(t)

# Compute Jacobian
dLdq = L.jacobian(q)
dLdqdot = L.jacobian(qdot)

# print('\033[1mEuler Lagrangian Equations')
EL = dLdqdot.diff(t) - dLdq

# LHS of the EL equations
lhs = sym.Matrix([exp for exp in EL])

# RHS of the EL equations
rhs = sym.Matrix([0 for exp in EL])

EL_eqns = sym.Eq(lhs, rhs)

solns = sym.solve(EL_eqns, qddot, dict=True)

# for sol_ in solns:
#     print('\n\033[1mSolution: ')
#     for q_ in qddot:
#         display(sym.simplify(sym.Eq(q_, sol_[q_])))
#     print("\n")

## Lamdifying the constant substituted solutions for theta1_ddot, theta2_ddot and theta3_ddot
# state_vec = [th1, qdot[0], th2, qdot[1], th3, qdot[2]]
# lambdified_solns = [sym.lambdify(state_vec, soln_[qddot_], modules='numpy') for soln_ in solns for qddot_ in qddot]
# th1ddot_func, th2ddot_func, th3ddot_func = lambdified_solns

```

**Lagrangian of the system:**



$$19.6 \cos(\theta_1(t) + \theta_2(t)) + 1.0 \cos(\theta_2(t) + \theta_3(t)) \left( \frac{d}{dt} \theta_1(t) \right)^2 + 1.0 \cos(\theta_2(t) + \theta_3(t)) \frac{d}{dt} \theta_1(t) \frac{d}{dt} \theta_2(t) + 1.0 \cos(\theta_2(t) + \theta_3(t)) \frac{d}{dt} \theta_1(t) \frac{d}{dt} \theta_3(t) + 9.8 \cos(\theta_3(t)) \frac{d}{dt} \theta_2(t) \frac{d}{dt} \theta_3(t) + 3.0 \left( \frac{d}{dt} \theta_1(t) \right)^2 + 3.0 \frac{d}{dt} \theta_1(t) \frac{d}{dt} \theta_2(t) + 1.0 \frac{d}{dt} \theta_1(t) \frac{d}{dt} \theta_3(t) + 1.5 \left( \frac{d}{dt} \theta_2(t) \right)^2 + 1.0 \frac{d}{dt} \theta_2(t) \frac{d}{dt} \theta_3(t) + 0.5 \left( \frac{d}{dt} \theta_3(t) \right)^2$$

```
In [55]: def th1ddot_func(theta1_k, theta1_dot_k, theta2_k, theta2_dot_k, theta3_k, theta3_dot_k):
    subs_ = {
        q[0]:theta1_k,
        q[1]:theta2_k,
        q[2]:theta3_k,
        qdot[0]:theta1_dot_k,
        qdot[1]:theta2_dot_k,
        qdot[2]:theta3_dot_k
    }

    return sym.N(solns[0][qddot[0]].subs(subs_))

def th2ddot_func(theta1_k, theta1_dot_k, theta2_k, theta2_dot_k, theta3_k, theta3_dot_k):
    subs_ = {
        q[0]:theta1_k,
        q[1]:theta2_k,
        q[2]:theta3_k,
        qdot[0]:theta1_dot_k,
        qdot[1]:theta2_dot_k,
        qdot[2]:theta3_dot_k
    }

    return sym.N(solns[0][qddot[1]].subs(subs_))

def th3ddot_func(theta1_k, theta1_dot_k, theta2_k, theta2_dot_k, theta3_k, theta3_dot_k):
    subs_ = {
        q[0]:theta1_k,
        q[1]:theta2_k,
        q[2]:theta3_k,
        qdot[0]:theta1_dot_k,
        qdot[1]:theta2_dot_k,
        qdot[2]:theta3_dot_k
    }

    return sym.N(solns[0][qddot[2]].subs(subs_))
```

```
In [56]: # Define the phi(q)
phi = x3 - 0
print("Phi")
display(phi)
print("\n")

dPhidq = sym.simplify(sym.Matrix([phi]).jacobian(q).T)
print("Gradient of the surface along q:")
display(dPhidq)
print("\n")

# Compute dL/dqdot
print("dL/dqdot")
display(dLdqdot)
print("\n")

# Define the hamiltonian
hamiltonian = sym.simplify(dLdqdot.T.dot(qdot)-lagrangian)
print("Hamiltonian:")
display(hamiltonian)
```

Phi

$$\sin(\theta_1(t) + \theta_2(t)) + \sin(\theta_1(t) + \theta_2(t) + \theta_3(t)) + \sin(\theta_1(t))$$

Gradient of the surface along q:

$$\begin{bmatrix} \cos(\theta_1(t) + \theta_2(t)) + \cos(\theta_1(t) + \theta_2(t) + \theta_3(t)) + \cos(\theta_1(t)) \\ \cos(\theta_1(t) + \theta_2(t)) + \cos(\theta_1(t) + \theta_2(t) + \theta_3(t)) \\ \cos(\theta_1(t) + \theta_2(t) + \theta_3(t)) \end{bmatrix}$$

dL/dqdot

$$\left[ 2.0 \cos(\theta_2(t) + \theta_3(t)) \frac{d}{dt} \theta_1(t) + 1.0 \cos(\theta_2(t) + \theta_3(t)) \frac{d}{dt} \theta_2(t) + 1.0 \cos(\theta_2(t) + \theta_3(t)) \frac{d}{dt} \theta_3(t) + 4.0 \cos(\theta_2(t)) \frac{d}{dt} \theta_1(t) + 2.0 \cos(\theta_2(t)) \frac{d}{dt} \theta_2(t) + 2.0 \cos(\theta_3(t)) \frac{d}{dt} \theta_2(t) \frac{d}{dt} \theta_3(t) + 3.0 \left( \frac{d}{dt} \theta_1(t) \right)^2 + 3.0 \frac{d}{dt} \theta_1(t) \frac{d}{dt} \theta_2(t) + 1.0 \frac{d}{dt} \theta_1(t) \frac{d}{dt} \theta_3(t) + 1.5 \left( \frac{d}{dt} \theta_2(t) \right)^2 + 1.0 \frac{d}{dt} \theta_2(t) \frac{d}{dt} \theta_3(t) + 0.5 \left( \frac{d}{dt} \theta_3(t) \right)^2 \right]$$

Hamiltonian:

$$-19.6 \cos(\theta_1(t) + \theta_2(t)) + 1.0 \cos(\theta_2(t) + \theta_3(t)) \left( \frac{d}{dt} \theta_1(t) \right)^2 + 1.0 \cos(\theta_2(t) + \theta_3(t)) \frac{d}{dt} \theta_1(t) \frac{d}{dt} \theta_2(t) + 1.0 \cos(\theta_2(t) + \theta_3(t)) \frac{d}{dt} \theta_1(t) \frac{d}{dt} \theta_3(t) - 9.8 \cos(\theta_3(t)) \frac{d}{dt} \theta_2(t) \frac{d}{dt} \theta_3(t) + 3.0 \left( \frac{d}{dt} \theta_1(t) \right)^2 + 3.0 \frac{d}{dt} \theta_1(t) \frac{d}{dt} \theta_2(t) + 1.0 \frac{d}{dt} \theta_1(t) \frac{d}{dt} \theta_3(t) + 1.5 \left( \frac{d}{dt} \theta_2(t) \right)^2 + 1.0 \frac{d}{dt} \theta_2(t) \frac{d}{dt} \theta_3(t) + 0.5 \left( \frac{d}{dt} \theta_3(t) \right)^2$$



```
In [57]: hamiltonain_func = sym.lambdify([th1, qdot[0], th2, qdot[1], th3, qdot[2]], hamiltonian)
```

## Problem 6 (10pts)

Similar to Problem 3, now you need to define dummy symbols for  $\dot{q}(\tau^+)$ , define the equations for impact update rules. Note that you don't need to solve the equations in this problem - in fact it's very time consuming to solve the analytical solution, and we will use a trick to get around it later!

**Turn in: Include a copy of the code used to define the equations for impact update and the code output (i.e. print out of the equations).**

```
In [58]: lambda_ = sym.symbols(r'\lambda')

th1_minus, th1dot_minus, th2_minus, th2dot_minus, th3_minus, th3dot_minus = \
sym.symbols(r'\theta1^-, \dot{\theta1}^-, \theta2^-, \dot{\theta2}^-, \theta3^-, \dot{\theta3}^-')

th1dot_plus, th2dot_plus, th3dot_plus = sym.symbols(r'\dot{\theta1}^+, \dot{\theta2}^+, \dot{\theta3}^+')

subs_minus = {
    q[0]:th1_minus,
    q[1]:th2_minus,
    q[2]:th3_minus,
    qdot[0]:th1dot_minus,
    qdot[1]:th2dot_minus,
    qdot[2]:th3dot_minus
}

subs_plus = {
    q[0]:th1_minus,
    q[1]:th2_minus,
    q[2]:th3_minus,
    qdot[0]:th1dot_plus,
    qdot[1]:th2dot_plus,
    qdot[2]:th3dot_plus
}

# display(qdot_plus)

# Momentum before and after impact
dLdqdot_minus = dLdqdot.subs(subs_minus)
dLdqdot_plus = dLdqdot.subs(subs_plus)

# Compute dPhidq and evaluate it at tau-
dPhidq_minus = dPhidq.subs(subs_minus)

# Compute hamiltonian at tau- and tau+
hamiltonian_minus = hamiltonian.subs(subs_minus)
hamiltonian_plus = hamiltonian.subs(subs_plus)

impact_eqns_lhs = sym.Matrix([ dLdqdot_plus[0] - dLdqdot_minus[0], dLdqdot_plus[1] - dLdqdot_minus[1], dLdqdot_plus
impact_eqns_rhs = sym.Matrix([ lambda_ * dPhidq_minus[0], lambda_ * dPhidq_minus[1], lambda_ * dPhidq_minus[2], 0 ])
impact_eqns = sym.Eq(impact_eqns_lhs, impact_eqns_rhs)
print('Equations for impact update:')
display(impact_eqns)
```

Equations for impact update:

$$\begin{bmatrix} 4.0\dot{\theta}_1^+ \cos(\theta_2^-) + 2.0\dot{\theta}_1^+ \cos(\theta_3^-) + 2.0\dot{\theta}_1^+ \cos(\theta_2^- + \theta_3^-) + 6.0\dot{\theta}_1^+ - 4.0\dot{\theta}_1^- \cos(\theta_2^-) - 2.0\dot{\theta}_1^- \cos(\theta_3^-) - 2.0\dot{\theta}_1^- \cos(\theta_2^- + \theta_3^-) - 6.0\dot{\theta}_1^- + \\ 2.0\dot{\theta}_1^+ \cos(\theta_2^-) + 2.0\dot{\theta}_1^+ \cos(\theta_3^-) + 1.0\dot{\theta}_1^+ \cos(\theta_2^- + \theta_3^-) + 3.0\dot{\theta}_1^+ - 2.0 \\ 1.0\dot{\theta}_1^+ \cos(\theta_3^-) + 1.0\dot{\theta}_1^+ \cos(\theta_2^- + \theta_3^-) \\ 2.0(\dot{\theta}_1^+)^2 \cos(\theta_2^-) + 1.0(\dot{\theta}_1^+)^2 \cos(\theta_3^-) + 1.0(\dot{\theta}_1^+)^2 \cos(\theta_2^- + \theta_3^-) + 3.0(\dot{\theta}_1^+)^2 + 2.0\dot{\theta}_1^+ \dot{\theta}_2^+ \cos(\theta_2^-) + 2.0\dot{\theta}_1^+ \dot{\theta}_2^+ \cos(\theta_3^-) + 1.0\dot{\theta}_1^+ \dot{\theta}_2^+ \cos \\ - 3.0\dot{\theta}_1^- \dot{\theta}_2^- - 1.0\dot{\theta}_1^- \dot{\theta}_3^- \cos(\theta_3^-) - 1.0\dot{\theta}_1^- \dot{\theta}_3^- \cos(\theta_2^- + \theta_3^-) - 1.0\dot{\theta}_1^- \dot{\theta}_2^- \cos(\theta_2^-) - 1.0\dot{\theta}_1^- \dot{\theta}_2^- \cos(\theta_3^-) - 1.0\dot{\theta}_1^- \dot{\theta}_2^- \cos(\theta_2^- + \theta_3^-) \end{bmatrix}$$

## Problem 7 (15pts)

Since solving the analytical symbolic solution of the impact update rules for the triple-pendulum system is too slow, here we will solve it along within the simulation. The idea is, when the impact happens, substitute the numerical values of  $q$  and  $\dot{q}$  at that moment into the equations you got in Problem 6, thus you will just need to solve a set equations with most terms being numerical values (which is very fast).

The first thing is to write a function called "impact\_update\_triple\_pend". This function at least takes in the current state of the system  $s(t^-) = [q(t^-), \dot{q}(t^-)]$  or  $\dot{q}(t^-)$ , inside the function you need to substitute in  $q(t^-)$  and  $\dot{q}(t^-)$ , solve for and return  $s(t^+) = [q(t^+), \dot{q}(t^+)]$  or  $\dot{q}(t^+)$  (which should be numerical values now). This function will replace lambdify, and you can use SymPy's "sym.N()" or "expr.evalf()" methods to convert SymPy expressions into numerical values. Test your function with  $\theta_1(\tau^-) = \theta_2(\tau^-) = \theta_3(\tau^-) = 0$  and  $\dot{\theta}_1(\tau^-) = \dot{\theta}_2(\tau^-) = \dot{\theta}_3(\tau^-) = -1$ .

Turn in: A copy of your "impact\_update\_triple\_pend" function, and the test result of your function.

```
In [59]: def impact_update_triple_pend(x):

    subs_values = {
        th1_minus:x[0],
        th2_minus:x[2],
        th3_minus:x[4],
        th1dot_minus:x[1],
        th2dot_minus:x[3],
        th3dot_minus:x[5]
    }

    impact_equations_subs = impact_eqns.subs(subs_values)
    # display(impact_equations_subs)
    # print("In impact_update_triple_pend")

    impact_solns = sym.solve(impact_equations_subs, [th1dot_plus, th2dot_plus, th3dot_plus, lambda_])
    # print(impact_solns)

    if(len(impact_solns) > 1):
        if(abs(impact_solns[0][3]) < 1e-2):
            # print("1")
            # print(np.array([x[0], impact_solns[1][0], x[2], impact_solns[1][1], x[4], impact_solns[1][2]]))
            return np.array([x[0], impact_solns[1][0], x[2], impact_solns[1][1], x[4], impact_solns[1][2]])
        else:
            # print("2")
            # print(np.array([x[0], impact_solns[0][0], x[2], impact_solns[0][1], x[4], impact_solns[0][2]]))
            return np.array([x[0], impact_solns[0][0], x[2], impact_solns[0][1], x[4], impact_solns[0][2]])
        else:
            # print("3")
            # print(np.array([x[0], impact_solns[0][0], x[2], impact_solns[0][1], x[4], impact_solns[0][2]]))
            return np.array([x[0], impact_solns[0][0], x[2], impact_solns[0][1], x[4], impact_solns[0][2]])

print("Test for the impact_update_triple_pend for the given input {}".format(impact_update_triple_pend([0, 1, 0, 1,
Test for the impact_update_triple_pend for the given input [0 1.000000000000000 0 1.000000000000000 0 -11.0000000000000
0]
```

## Problem 8 (15pts)

Similar to the single-pendulum system, you will still want to implement a function named "impact\_condition\_triple\_pend" to indicate the moment when impact happens. Again, you need to use the constraint  $\phi$ . After obtaining the impact condition function, simulate the triple-pendulum system with impact for  $t \in [0, 2]$ ,  $dt = 0.01$  with initial condition  $\theta_1 = \frac{\pi}{3}, \theta_2 = \frac{\pi}{3}, \theta_3 = -\frac{\pi}{3}$  and  $\dot{\theta}_1 = \dot{\theta}_2 = \dot{\theta}_3 = 0$ . Plot the simulated trajectory versus time and animate your simulated trajectory.

Hint 1: You will need to modify the simulate function!

Turn in: A copy of code for the impact update function and simulate function, as well as code output including the plot of simulated trajectory and the animation. The video should be uploaded separately from the .pdf file through Canvas, and it should be in ".mp4" format. You can use screen capture or record the screen directly with your phone.

```
In [60]: phi_func = sym.lambdify([th1, th1_dot, th2, th2_dot, th3, th3_dot], phi)

def impact_condition_triple_pend(x, threshold=1e-1):
    th1 = x[0]
    th2 = x[2]
    th3 = x[4]
    th1_dot = x[1]
    th2_dot = x[3]
    th3_dot = x[5]
    phi_eval = phi_func(th1, th1_dot, th2, th2_dot, th3, th3_dot)

    if(abs(phi_eval) < threshold):
        return True

    return False

# define extended dynamics for the triple pendulum system
def triple_pendulum_dyn(x):
    """
    System dynamics function (extended)

    Parameters
    =====
    x: NumPy array
    x = [th1, th2, th3, th1dot th2dot, th3dot] is the extended system
    state vector, including the position and
    the velocity of the particle

    Return
    =====
```

```

xdot: NumPy array
time derivative of input state vector,
xdot = [th1dot th2dot, th3dot, th1ddot, th2ddot, th3ddot]

"""
theta1_k = x[0]
theta2_k = x[2]
theta3_k = x[4]
theta1_dot_k = x[1]
theta2_dot_k = x[3]
theta3_dot_k = x[5]

theta1_ddot_k = th1ddot_func(theta1_k, theta1_dot_k, theta2_k, theta2_dot_k, theta3_k, theta3_dot_k)
theta2_ddot_k = th2ddot_func(theta1_k, theta1_dot_k, theta2_k, theta2_dot_k, theta3_k, theta3_dot_k)
theta3_ddot_k = th3ddot_func(theta1_k, theta1_dot_k, theta2_k, theta2_dot_k, theta3_k, theta3_dot_k)

return np.array([theta1_dot_k, theta1_ddot_k, theta2_dot_k, theta2_ddot_k, theta3_dot_k, theta3_ddot_k])

# define simulate function
def simulate_triple_pend_with_impact(f, x0, tspan, dt, integrate):

    N = int((max(tspan) - min(tspan)) / dt)
    x = np.copy(x0)
    tvec = np.linspace(min(tspan), max(tspan), N)
    xtraj = np.zeros((len(x0), N))
    impact_eval = False
    for i in range(N):
        if impact_condition_triple_pend(x) is True and not impact_eval:
            x = impact_update_triple_pend(x)
            xtraj[:, i] = integrate(f, x, dt)
            impact_eval = True
        else:
            xtraj[:, i] = integrate(f, x, dt)
            impact_eval = False
        x = np.copy(xtraj[:, i])

    return xtraj

# Simulation parameters
t0 = 0
tf = 2.0
dt = 0.01
N = int((tf - t0)/dt)
tvec = np.linspace(t0, tf, N)

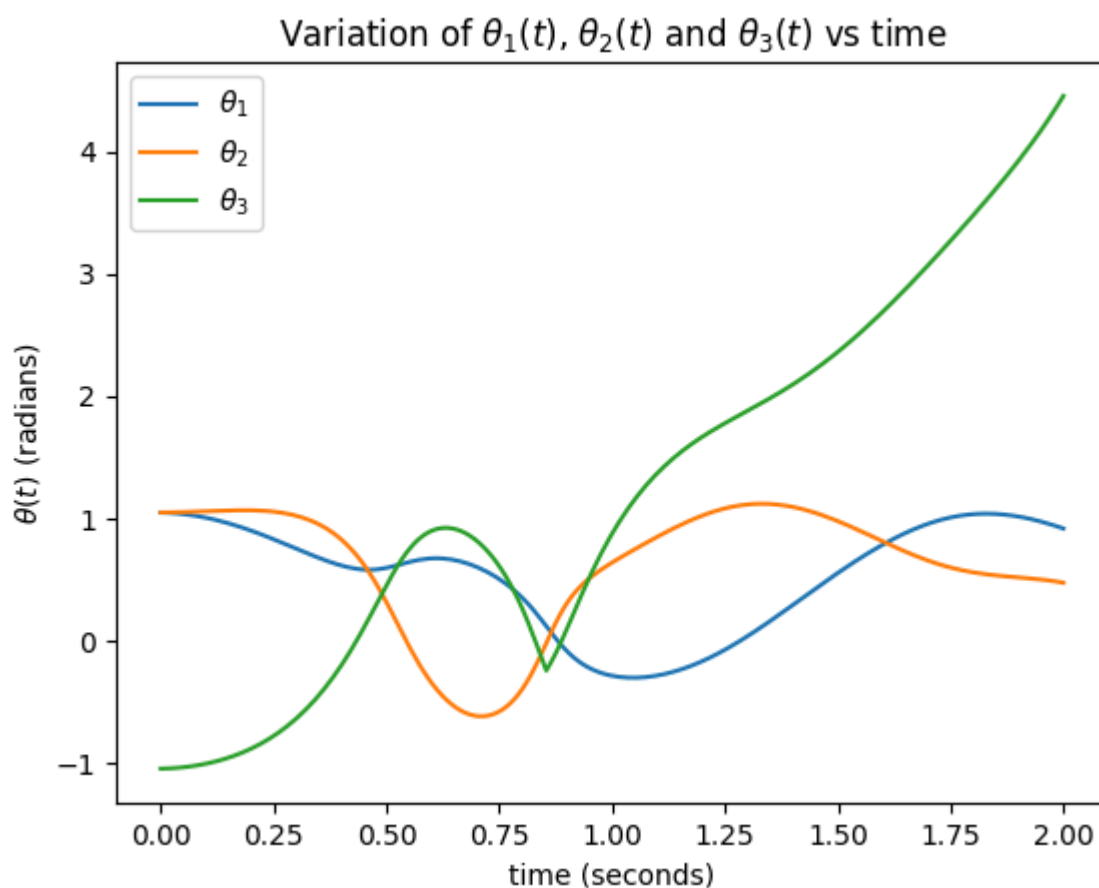
# initial conditions
x0 = np.array([np.pi/3, 0, np.pi/3, 0, -np.pi/3, 0])
# simulate the trajectory
x_traj = simulate_triple_pend_with_impact(triple_pendulum_dyn, x0, tspan=[t0, tf], dt=dt, integrate=integrate)

```

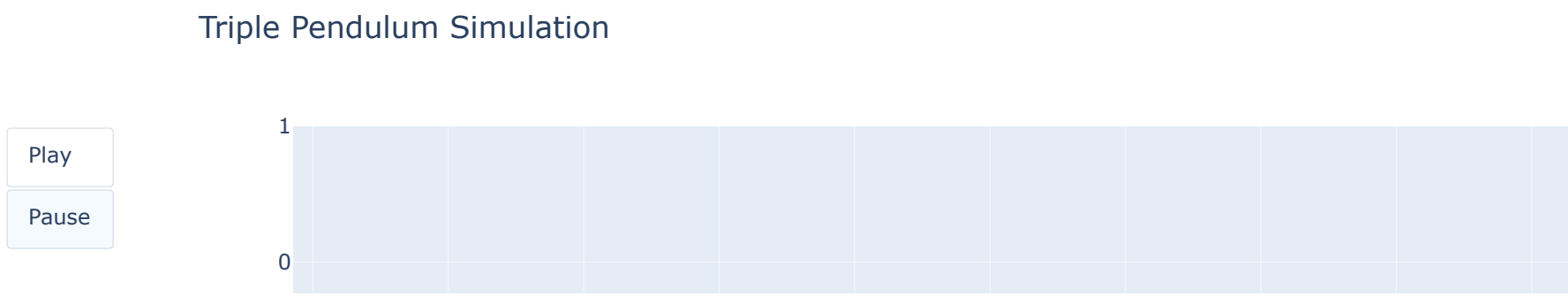
```

In [64]: # plot
plt.plot(tvec, x_traj[0,:], label=r"$\theta_1$")
plt.plot(tvec, x_traj[2,:], label=r"$\theta_2$")
plt.plot(tvec, x_traj[4,:], label=r"$\theta_3$")
plt.title(r'Variation of $\theta_1(t)$, $\theta_2(t)$ and $\theta_3(t)$ vs time')
plt.xlabel("time (seconds)")
plt.ylabel(r"$\theta(t)$ (radians)")
plt.legend()
plt.show()

```



```
In [69]: animate_triple_pend(np.delete(x_traj, [1,3,5], axis=0), L1=1, L2=1, L3=1, T=tf-t0)
```



### Problem 9 (5pts)

Compute and plot the Hamiltonian of the simulated trajectory for the triple-pendulum system with impact.

Turn in: A copy of code used to compute the Hamiltonian, also include the code output, which should the plot of the Hamiltonian versus time.

```
In [63]: def compute_hamiltonian_traj(x_traj):
    hamiltonian_traj = []
    for col in range(x_traj.shape[1]):
        th1_k, th1_dot_k, th2_k, th2_dot_k, th3, th3_dot_k = x_traj[:, col]
        # print(x_traj[:, col])
        h_k = hamiltonain_func(th1_k, th1_dot_k, th2_k, th2_dot_k, th3, th3_dot_k)
        hamiltonian_traj.append(h_k)

    return np.array(hamiltonian_traj)

hamiltonian_traj = compute_hamiltonian_traj(x_traj)

# Set title and labels
plt.title(r"Hamiltonian evolution with time")
plt.xlabel("time (in sec)")
plt.ylabel("Hamiltonian (KE + PE)")
plt.grid(True)
plt.ticklabel_format(style='plain', axis='y', useOffset=False)

plt.plot(tvec, hamiltonian_traj[:])
```

```
Out[63]: [<matplotlib.lines.Line2D at 0x7ee4fedf52a0>]
```

